

DVWA Setup, SQL Injection & Mitigation Guide

A practical cybersecurity walk-through on Kali Linux for ethical hacking and secure coding practices.

1. Installing DVWA on Kali Linux

Follow these sequential steps to install and configure the Damn Vulnerable Web Application (DVWA) on Kali Linux.

1 Start Required Services

Ensure that the Apache web server and MariaDB database services are active:

```
sudo systemctl start apache2
sudo systemctl start mariadb
```

2 Clone and Deploy DVWA

Clone the official repository directly into the Apache web root directory and adjust permissions:

```
cd /var/www/html
sudo git clone https://github.com/digininja/DVWA.git
sudo chown -R www-data:www-data /var/www/html/DVWA
sudo chmod -R 755 /var/www/html/DVWA
```

3 Configure Database Credentials

Create the active configuration file from the distributed sample and edit database connection settings:

```
cd /var/www/html/DVWA/config
sudo cp config.inc.php.dist config.inc.php
sudo nano config.inc.php
```

Update the configuration variables as follows:

```
$_DVWA[ 'db_user' ]      = 'dvwa';
$_DVWA[ 'db_password' ] = 'password';
$_DVWA[ 'recaptcha_public_key' ] = '';
$_DVWA[ 'recaptcha_private_key' ] = '';
```

4 Create MariaDB User & Database

Log into MariaDB console to initialize the web application database and set appropriate grants:

```
sudo mariadb -u root
```

Execute the following database statements:

```
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
```

```
FLUSH PRIVILEGES;  
EXIT;
```

5 Initialize & Access Application

Complete the setup process via your browser:

- Navigate to `http://127.0.0.1/DVWA/setup.php` and click **Create / Reset Database** at the bottom.
- Log in at `http://127.0.0.1/DVWA/login.php` using credentials:
 - **Username:** `admin`
 - **Password:** `password`

2. Executing SQL Injection (Low Security)

Set the target security level to **Low** under the **DVWA Security** tab, then navigate to the **SQL Injection** section.

Vulnerable Backend Implementation

The application concatenates user input directly into the SQL string without sanitization or parameterization:

```
$id = $_GET['id'];  
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";  
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query);
```

Exploitation Payload

Enter the following payload into the **User ID** input field to extract usernames and hashed passwords from the database:

```
' UNION SELECT user, password FROM users #
```

Payload Breakdown

- `'` : Closes the original string literal in the query structure.
- `UNION SELECT user, password FROM users` : Appends a secondary SQL command to extract credentials from the `users` table.
- `#` : Comments out remaining syntax in the original query to prevent syntax errors.

Extracted Results Overview

User ID	Username	Password Hash (MD5)
1	admin	<code>5f4dcc3b5aa765d61d8327deb882cf99</code> (password)
2	gordonb	<code>e99a18c428cb38d5f260853678922e03</code> (abc123)
3	1337	<code>8d3533d3ae1c13a357d2946d4272b5bd</code> (charley)

3. Prevention Using Prepared Statements

SQL Injection vulnerabilities occur when dynamic SQL queries conflate input data with command logic. Prepared Statements (Parameterized Queries) guarantee that user input is treated strictly as data parameters.

Code Comparison

Vulnerable Code (Dynamic Concatenation)

```
// UNSAFE: Direct string interpolation allows payload execution
$id = $_GET['id'];
$sql = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = $pdo->query($sql);
```

Secure Code (PDO Prepared Statements)

```
$id = $_GET['id'];

// 1. Send SQL template structure with parameter placeholders
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = :id');

// 2. Bind parameter values explicitly and execute
$stmt->execute(['id' => $id]);

// 3. Retrieve results securely
$results = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Why Prepared Statements Work

With prepared statements, the database engine compiles the SQL query structure first. When parameters are sent subsequently, any injected SQL syntax (such as `' UNION SELECT ... # )` is interpreted purely as a plain string literal value for `user_id`, neutralizing the payload completely.